```
    def person_params
      params.require(:person).permit(:name, :age)
    end
end
```

4.5.1 Permitted Scalar Values

Given

```
params.permit(:id)
```

the key `:id` will pass the whitelisting if it appears in `params` and it has a permitted scalar value associated. Otherwise the key is going to be filtered out, so arrays, hashes, or any other objects cannot be injected.
The permitted scalar types
are `String`, `Symbol`, `NilClass`, `Numeric`, `TrueClass`, `FalseClass`, `Date`,`Time`, `DateTime`, `StringIO`, `IO`, `ActionDispatch::Http::UploadedFile` and `Rack::Test::UploadedFile`.
To declare that the value in `params` must be an array of permitted scalar values map the key to an empty array:

```
params.permit(id: [])
```

To whitelist an entire hash of parameters, the `permit!` method can be used:

```
params.require(:log_entry).permit!
```

This will mark the `:log_entry` parameters hash and any sub-hash of it permitted. Extreme care should be taken when using `permit!` as it will allow all current and future model attributes to be mass-assigned.

4.5.2 Nested Parameters

You can also use permit on nested parameters, like:

```
params.permit(:name, { emails: [] },
              friends: [ :name,
                         { family: [ :name ], hobbies: [] }])
```

This declaration whitelists the `name`, `emails` and `friends` attributes. It is expected that `emails` will be an array of permitted scalar values and that `friends` will be an array of resources with specific attributes : they should have a `name` attribute (any permitted scalar values allowed), a `hobbies`attribute as an array of permitted scalar values, and a `family` attribute which is restricted to having a `name` (any permitted scalar values allowed, too).

4.5.3 More Examples

You want to also use the permitted attributes in the `new` action. This raises the problem that you can't use `require` on the root key because normally it does not exist when calling `new`:

```
# using `fetch` you can supply a default and use
# the Strong Parameters API from there.
params.fetch(:blog, {}).permit(:title, :author)
```

`accepts_nested_attributes_for` allows you to update and destroy associated records. This is based on the `id` and `_destroy` parameters:

```
# permit :id and :_destroy
params.require(:author).permit(:name, books_attributes: [:title, :id,
:_destroy])
```

Hashes with integer keys are treated differently and you can declare the attributes as if they were direct children. You get these kinds of parameters when you use `accepts_nested_attributes_for`in combination with a `has_many` association:

```
# To whitelist the following data:
# {"book" => {"title" => "Some Book",
#             "chapters_attributes" => { "1" => {"title" => "First
Chapter"},
```